

FRI

UNIVERZA V LJUBLJANI | Fakulteta za računalništvo in informatiko

Dodatne vsebine pri predmetu Osnove programiranja

Ana Halužan Vasle

Verzija 1.0

Študijsko leto 2026/2027

Izjava o avtorstvu in uporabi velikih jezikovnih modelov

Pričujoča skripta je v avtorsko delo Ane Halužan Vasle, nastalo v okviru izvajanja vaj pri predmetu Osnove programiranja na Fakulteti za kemijo in kemijsko tehnologije Univerze v Ljubljani.

Pri pripravi Dodatnih vsebin so bili uporabljeni veliki jezikovni modeli (Claude in ChatGPT) za zasnovo obsega in strukture poglavij, jezikovno urejanje, izboljšanje izražanja in kot pomoč pri urejanju oblikovanja v LaTeX-u. Predlagano vsebino je avtorica kritično ovrednotila, preverila in v veliki meri preoblikovala ter napisala na novo.

Kazalo

1. Markdown **7**
1.1 Osnovna pravila oblikovanja 7
1.2 Pregled osnovnih znakov za oblikovanje 8

2. Git **9**
2.1 Namestitev in osnovna nastavitev 9
2.2 Osnovni delovni tok 10
2.3 Pregled osnovnih ukazov 12

3. Viri **13**
3.1 Markdown 13
3.2 Git 13

1. Markdown

Markdown je preprost označevalni jezik (angl. *markup language*), za katerega je značilna preprosta sintaksa, ki omogoča, da besedilo ostane berljivo tudi med urejanjem. Datoteke, zapisane v Markdownu, imajo končnico `.md` in jih lahko odpremo v kateremkoli urejevalniku besedila.

Markdown je med programerji razširjen predvsem za datoteke `README` in druge datoteke, ki spremljajo objavljeno kodo, na primer opise napak (angl. *issues*) in predloge sprememb, ter za tehnično dokumentacijo API-jev. Njegova sintaksa je podprta tudi v številnih komunikacijskih orodjih, kot so Slack, Discord in Microsoft Teams. Kot osnovni format zapisa se uporablja tudi v orodjih za zapiske, kot sta Obsidian in Notion, prav tako so z Markdownom pogosto oblikovani izpisi umetne inteligence.

1.1 Osnovna pravila oblikovanja

Pred besedilo, ki ga želimo zapisati v obliki naslova, zapišemo enega ali več znakov `#`. Več kot imamo znakov, več ravni naslovov smo ustvarili:

```
# Naslov 1. reda
## Naslov 2. reda
### Naslov 3. reda
```

Besedilo zapišemo ležeče tako, da ga obdamo z enojnimi zvezdicami ali podčrtaji. Krepko poudarjen zapis dosežemo z dvojnimi zvezdicami ali podčrtaji. Če želimo, da je besedilo zapisano krepko in ležeče, pa dodamo trojne zvezdice.

```
*lezece besedilo* ali _lezece besedilo_
**krepko besedilo** ali __krepko besedilo__
***krepko in lezece hkrati***
```

Seznam po alinejah označujemo z znakom `-`, `*` ali `+`:

```
- Kava
- Caj
- Mleko
```

Če želimo pripraviti oštevilčeni seznam, pred vsako točko zapišemo zaporedno številko s piko:

```
1. Vstani
2. Skuhaj kavo
3. Zacni programirati
```

Povezavo do spletne strani zapišemo tako, da besedilo povezave damo v oglate, naslov strani pa v okrogle oklepaje, ki stično sledijo oglatim: `[Spletna stran FRI](https://www.fri.uni-lj.si)`.

Če v besedilo vstavljamo sliko, pred pred oglati oklepaj dodamo še klicaj:
`![opis slike](ime-slike.jpg)`

Kodo znotraj besedila označimo z enojnimi obrnjenimi narekovaji (angl. *backticks*): `, v primeru, da imamo več vrstic kode, pa pred in za njo uporabimo tri obrnjene narekovaje[1].

```
```
def pozdrav():
    print("Zivjo, svet!")
```
```

Citat oziroma poudarjen odstavek zapišemo tako, da pred besedilo dodamo znak >.

Trije zaporedni vezaji (---) ali tri zaporedne zvezdice (***), zapisane v svoji vrstici, tvorijo vodoravno črto.

1.2 Pregled osnovnih znakov za oblikovanje

Najpogosteje uporabljeni znaki za oblikovanje Markdown so zbrani v Tabeli 1.

Znak:	**Opis:**
#	naslov
* ali _	ležeče besedilo
** ali __	krepko besedilo
***	krepko in ležeče hkrati
-	alineja
X.	oštevilčen seznam
[besedilo](url)	hiperpovezava
![besedilo](slika.jpg)	vstavljanje slike
`	koda znotraj besedila
``	večvrstična koda
>	citat
--- ali ***	vodoravna črta

Tabela 1: Pogosto uporabljeni osnovni znaki za oblikovanje Markdown.

[1]Podobno kot pri ustvarjanju večvrstičnih komentarjev v Pythonu, le da tam uporabimo trojne enojne ali dvojne narekovaje.

2. Git

Git je sistem za nadzor različic kode (angl. *version control system*), ki sledi spremembam datotek skozi čas. Git se izvaja lokalno na našem računalniku in za delovanje ne potrebuje interneta.

GitHub je spletna storitev, ki gosti repozitorije Git (angl. *repository*) na strežniku, kar omogoča sodelovanje z drugimi, deljenje kode in varnostno kopijo projekta. Repozitoriji Git so mape, v katerih se shranjujejo spremembe datotek.

Git in GitHub sta ločena: Git je orodje, GitHub pa eden od spletnih ponudnikov, ki to orodje uporablja. Med ponudniki, ki prav tako uporabljajo Git sta še npr. GitLab in Bitbucket.

Vsako shranjeno stanje projekta v Git (t.i. *commit*) ostane trajno zabeleženo, zato se lahko kadarkoli vrnemo k prejšnji verziji kode, primerjamo spremembe med verzijami ali spremljamo spremembe sodelavcev. Če je projekt poleg lokalnega računalnika shranjen tudi na GitHubu, imamo varnostno kopijo, hkrati pa lahko pri razvoju kode sočasno sodeluje več sodelavcev istočasno. V tem primeru lahko novo funkcionalnost programa razvijamo ločeno od glavne, delujoče kode, na t.i. vejah (angl. *branches*), in jih z glavno združimo šele, ko je veja dokončana in preizkušena.

Git je danes privzeto orodje za sodelovanje pri programerskih projektih, zato je poznavanje njegovih osnov pomembno. V številnih orodjih za razvijanje kode, kot sta VS Code in PyCharm, sta Git in dostop do GitHuba že vključena, kar nam olajša delo, saj ne potrebujemo ukazne vrstice.

2.1 Namestitev in osnovna nastavitev

Git namestimo s spletne strani `https://git-scm.com/downloads`, kjer izberemo različico, ki je združljiva z našim operacijskim sistemom. Po namestitvi Git uporabljamo prek ukazne vrstice (*terminal* za macOS oz. *command line* za Windows ali Linux).

Preden ga začnemo uporabljati, moramo nastaviti svoje uporabniško ime in e-poštni naslov, ki se bosta izpisala ob vsakem *commitu*. To storimo v terminalu z ukazoma `git config --global user.name` in `git config --global user.email`, ki jima sledijo podatki o imenu in naslovu, zapisani v narekovajih:

```
git config --global user.name "Ime Priimek"
git config --global user.email "ime.priimek@mail.si"
```

2.2 Osnovni delovni tok

2.2.1 Ustvarjanje repozitorija

Repozitorij lahko ustvarimo na dva načina: z ukazom `git init`, s katerim iz obstoječe mape na računalniku naredimo Git repozitorij, ali z ukazom `git clone`, s katerim na računalnik prenesemo že obstoječi repozitorij z GitHuba. Če želimo obstoječo mapo spremeniti v repozitorij Git, moramo najprej premakniti svoj delovni direktorij v izbrano mapo. To storimo ukazom:

```
cd pot/do/mape
git init
```

Če mapa še ne obstaja, jo lahko navedemo kot argument in Git jo bo ustvaril v delovnem direktoriju:

```
cd pot/do/lokacije
git init ime-mape
```

V primeru uporabe `git clone` moramo navesti povezavo do želenega repozitorija:

```
git init
git clone https://github.com/uporabnik/ime-repozitorija.git
```

2.2.2 Sledenje spremembam

Ko datoteko ustvarimo ali spremenimo, jo moramo najprej označiti, da lahko Git sledi njenim spremembam. Označevanju pravimo *stage*, trajni zabeležki spremembe pa *commit*. Označevanje izvedemo z ukazom `git add`. Ukazu `git commit -m` dodamo kratek opis sprememb v obliki niza. Opis naj bo kratek in jasen glede sprememb v dani verziji, saj to olajša iskanje po zgodovini projekta.

```
git add ime_datoteke.py
git commit -m "Dodana funkcija za izracun povprecja"
```

Z ukazom `git add .` označimo vse spremenjene datoteke v mapi naenkrat.

2.2.3 Pošiljanje in prejemanje sprememb z GitHuba

Ko so spremembe zabeležene lokalno, jih z ukazom `git push` pošljemo na GitHub. Če želimo prenesti spremembe, ki so jih na GitHub naložili drugi, to storimo z ukazom `git pull`:

```
git push
git pull
```

Nasvet: preden začnete z nadaljnjim delom na projektu z več udeleženci, najprej izvedite `git pull`, saj se tako izognete sporom med različicami (angl. *merge conflicts*), do katerih pride, če več ljudi neodvisno spremeni iste vrstice kode.

2.2.4 Preverjanje stanja in zgodovine

Trenutno stanje repozitorija, ki vključuje imena datoteke in njihovo stanje - t.j. ali so spremenjene, označene ali še neoznačene za*commit*), preverimo z ukazom `git status`. Zgodovino vseh *commitov* pa pridobimo z ukazom `git log`.

```
git status
git log
```

2.2.5 Veje (angl. branches)

Glavna, delujoča različica kode je običajno poimenovana `main` ali `master`. Z ustvarjanjem veje (ukaz `git branch`) naredimo neodvisno kopijo glavne različice, na kateri lahko razvijamo novo funkcionalnost brez tveganja, da nepopravljivo spremenimo/pokvarimo glavno kodo. Na vejo preklopimo z ukazom `git checkout`. Ko je sprememba na veji preizkušena in deluje, se pomaknemo nazaj v glavno različico (ukaz `git checkout`, novo funkcionalnost pa z ukazom `git merge` združimo nazaj v glavno vejo:

```
git branch nova-funkcionalnost
git checkout nova-funkcionalnost
git checkout main
git merge nova-funkcionalnost
```

Ukaza `git branch` in `git checkout` lahko združimo v enega, ki novo vejo hkrati ustvari in nanjo tudi preklopi:

```
git checkout -b nova-funkcionalnost
```

Nasvet: datoteke, za katere ne želimo, da jih Git spremlja naštejemo v datoteki `.gitignore`, ki jo shranimo v korensko mapo repozitorija. Git bo pri sledenju spremembam navedene datoteke preskočil.

2.3 Pregled osnovnih ukazov

Najpogosteje uporabljeni ukazi Git so zbrani v Tabeli 1[2].

Ukaz:	**Opis:**
`git init`	ustvari nov repozitorij v trenutni mapi
`git clone <url>`	prenese obstoječi repozitorij na računalnik
`git status`	prikaže trenutno stanje repozitorija
`git add <datoteka>`	označi datoteko za naslednji *commit*
`git commit -m "..."`	trajno zabeleži označene spremembe
`git push`	pošlje lokalne *commite* na GitHub
`git pull`	prenese spremembe z GitHuba na računalnik
`git log`	prikaže zgodovino *commitov*
`git branch <ime>`	ustvari novo vejo
`git checkout <ime>`	preklopi na izbrano vejo
`git merge <ime>`	združi izbrano vejo z trenutno vejo

Tabela 2: Pogosto uporabljeni osnovni ukazi Git.

[2]Pregled vseh ukazov je na voljo na: https://git-scm.com/cheat-sheet

3. Viri

3.1 Markdown

Mardown. (22. 9. 2026). V *Wikipedia: free encyclopedia*. https://en.wikipedia.org/wiki/Markdown

Gruber, J. (b.d.). *Markdown: Syntax*. https://daringfireball.net/projects/markdown/syntax

Leonard, S. (2016). *Guidance on Markdown: Design Philosophies, Stability Strategies, and Select Registrations*. RFC Editor. https://doi.org/10.17487/rfc7764

3.2 Git

Chacon, S., Straub, B. (2014). *Pro Git (Second edition.)*. Apress. https://doi.org/10.1007/978-1-4842-0076-6